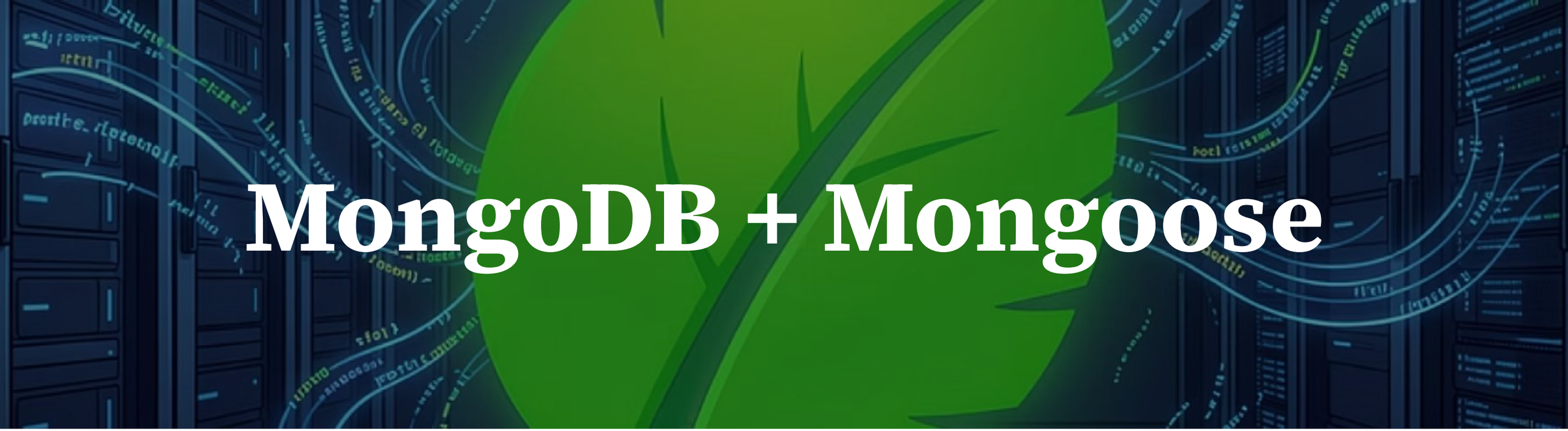


MongoDB + Mongoose

MERN Integration & Deployment

MODULE 5 · DAY 5 – BACKEND DATA LAYER

Build and integrate a database with a full MERN stack application over 6 focused hours.

Made with GAMMA

OVERVIEW

What is MongoDB?

MongoDB is a NoSQL database that stores data in a flexible, JSON-like format (BSON). Unlike traditional relational databases, it requires no strict table structure and is designed for scalability and high performance.

NoSQL Database

Stores data as BSON documents — a binary, JSON-like format — rather than rows and columns.

Flexible Schema

No rigid table structure required. Documents in the same collection can have different fields.

High Performance

Designed for scalability and speed, supporting horizontal scaling across distributed systems.

Example Document

A record like `{ name: "John", age: 25, skills: ["JavaScript", "React"] }` is a single MongoDB document.

Made with GAMMA

CONCEPTS

SQL vs NoSQL

Understanding the key differences between relational and non-relational databases helps you choose the right tool. MongoDB excels for dynamic applications like e-commerce and social platforms.

Feature	SQL (Relational)	NoSQL (MongoDB)
Structure	Tables	Collections
Schema	Fixed	Flexible
Scaling	Vertical	Horizontal
Query Language	SQL	JSON-like operators
Relationships	Foreign keys & joins	Embedded or referenced

Mapping SQL to MongoDB: database → database, tables → collections, rows → documents (BSON), columns → fields.

Made with GAMMA

STRUCTURE

MongoDB Core Concepts

MongoDB organizes data in a clear three-level hierarchy. Understanding this structure is the foundation for all database operations in your MERN application.

Database

The top-level container. Contains one or more collections. Example: `ecommerce`

Collection

A group of related documents, analogous to a table. Example: `products`

Document

A single JSON-like record. Example: `{name: "Shirt", price: 999}`

Hierarchy: **Database** → **Collection** → **Document**

OPERATIONS

MongoDB CRUD Operations

CRUD stands for Create, Read, Update, and Delete — the four fundamental operations for interacting with any database. MongoDB provides dedicated shell commands for each.

Create

Methods:

`insert(), save()`

Shell Command:

```
db.users.insertOne({
name: "Parth"
})
```

Read

Methods:

`find(), findOne()`

Shell Command:

```
db.users.find()
```

Update

Methods:

`updateOne(), updateMany()`

Shell Command:

```
db.users.updateOne(
{name:"Parth"},
{$set:{age:25}}
)
```

Delete

Methods:

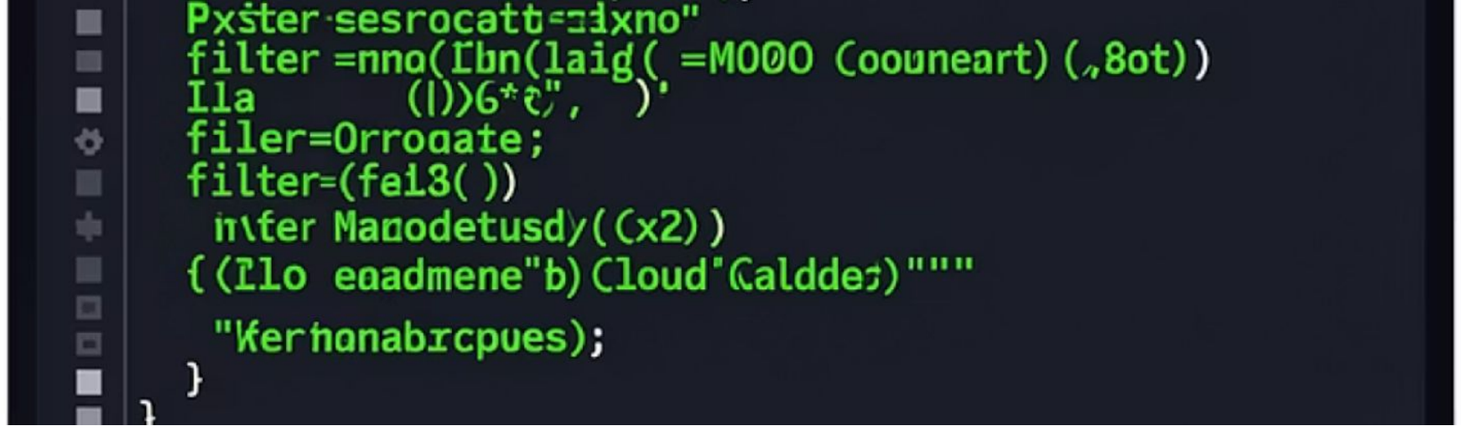
`deleteOne(), deleteMany()`

Shell Command:

```
db.users.deleteOne(
{name:"Parth"}
)
```

Practice all four operations in the MongoDB Shell before moving to Mongoose. Mastering raw commands builds intuition for ODM behavior.

Made with GAMMA



QUERIES

Advanced Queries in MongoDB

Beyond basic CRUD, MongoDB supports powerful query operators for filtering, sorting, and limiting results — essential for building real-world features like product filtering by price or category.

Query Operators

Equality: `db.users.find({age: 25})`
Greater Than ($gt): `db.users.find({age: {$gt: 20}})`
Less Than ($lt): `db.users.find({age: {$lt: 30}})`
In Array ($in): `db.users.find({role: {$in: ["admin","user"]}})`

Sorting & Limiting

Sort Descending: `db.users.find().sort({age: -1})`
Sort Ascending: `db.users.find().sort({age: 1})`
Limit Results: `db.users.find().limit(5)`

Use Case: Filter products by price range or category in an e-commerce app.

Made with GAMMA

ODM LAYER

What is Mongoose?

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It adds a structured, schema-based layer on top of the raw MongoDB driver, resulting in cleaner, more maintainable backend code.

Data Validation

Enforce required fields, types, and constraints before data ever touches the database.

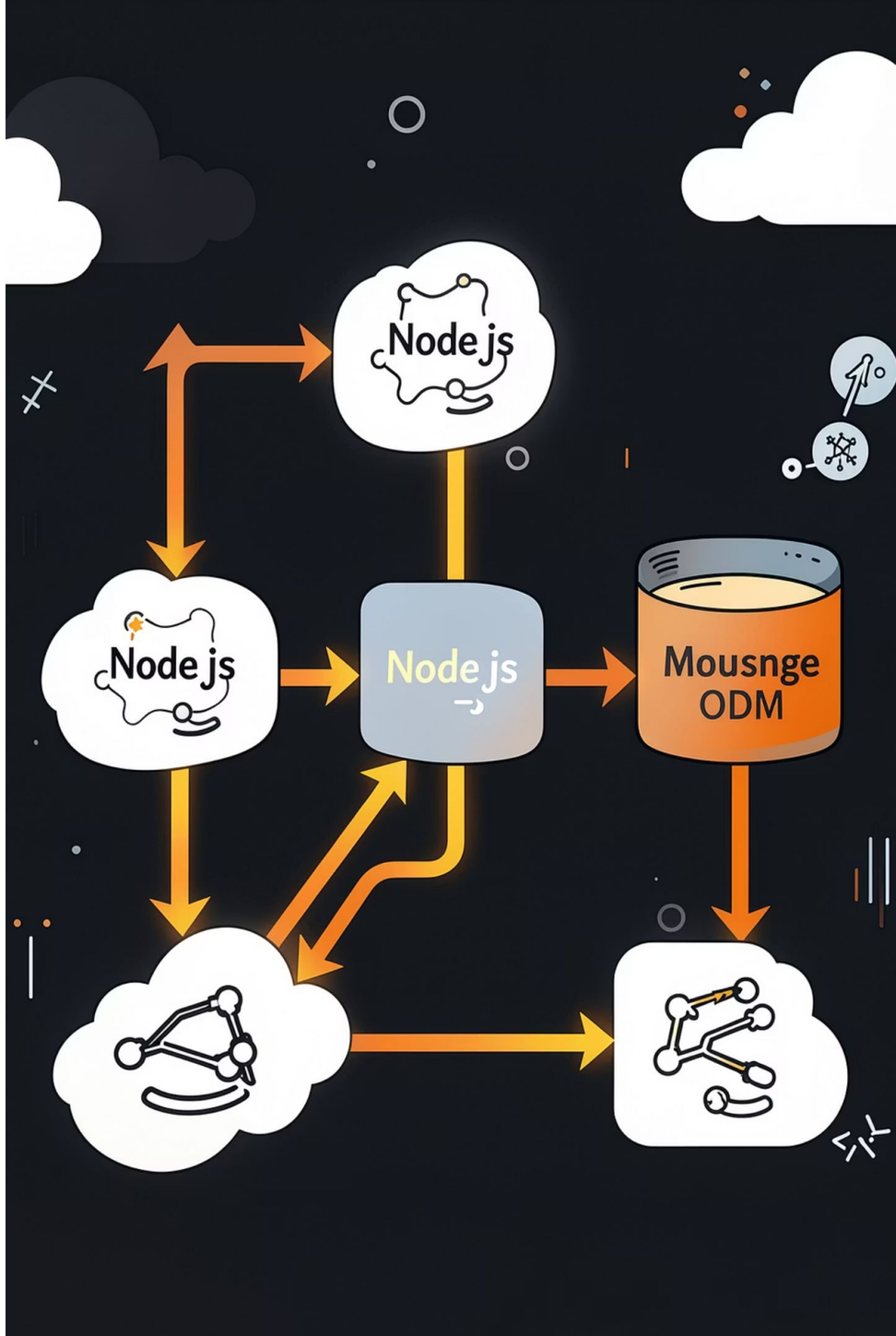
Schema-Based Design

Define the shape of your documents up front, making your data model explicit and predictable.

Cleaner Code

Mongoose replaces verbose MongoDB driver calls with expressive, readable JavaScript models and methods.

Flow: **Node.js → Mongoose (Object Mapping) → Mongo Driver → MongoDB**



SETUP

Mongoose Setup, Schema & Validation

Getting Mongoose connected and your first schema defined are the two critical steps to a working data layer. Once connected, schemas define the structure and validation rules for every document your app creates.

Installation & Connection

```
npm install mongoose
```

```
const mongoose = require('mongoose');
mongoose.connect(
  'mongodb://127.0.0.1:27017/testDB'
)
.then(() => console.log("Connected"))
.catch(err => console.log(err));
```

Schema, Model & Validation

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: {
    type: String,
    required: true
  }
});

const User = mongoose.model(
  'User', userSchema
);
```

Schema

Defines the structure and field types of a document.

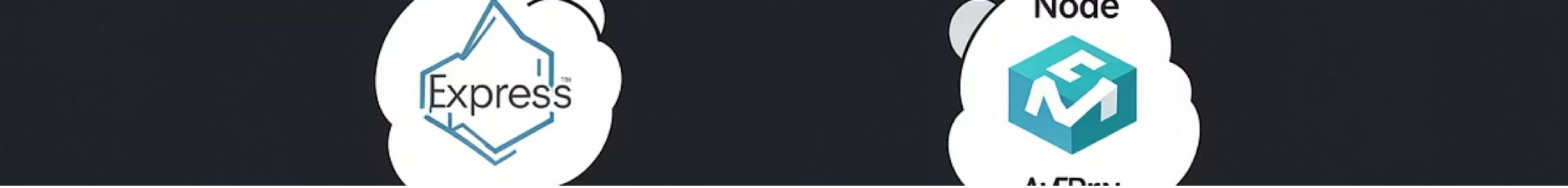
Model

The JavaScript interface used to query and write to the collection.

Validation

Ensures only correct, complete data is saved to the database.

Made with GAMMA



INTEGRATION

MERN Stack Integration

With MongoDB and Mongoose in place, the full MERN stack is connected. Data flows from the React front end, through Express routes and Node.js middleware, to MongoDB — and back — in a seamless loop.

1. React Front End

The journey begins here. Users interact with the UI to **Submit Forms** and trigger actions.

2. Express Middleware

Routes receive the request. Express and Node.js **Handle Requests** and manage logic.

3. MongoDB Persistence

Mongoose interacts with the database to **Store & Retrieve** data efficiently.

4. Full Cycle Loop

The server **Sends a Response** back to the client, completing the seamless loop.

Every layer has a single responsibility: React manages UI, Express handles routing, Node runs the server, and MongoDB persists data via Mongoose models.

Made with GAMMA

OUTCOME

Mini Task: Build a Product API

By the end of Day 5, students will have a fully operational CRUD backend connected to MongoDB via Mongoose, wired into their MERN application, and ready for deployment.

Add Product

Create a POST route using `Product.create(req.body)` to insert a new product document into MongoDB.

View Products

Build a GET route using `Product.find()` to return all products to the React front end.

Update Price

Wire a PUT route using `Product.findByIdAndUpdate()` to modify an existing product's price field.

Delete Product

Implement a DELETE route using `Product.findByIdAndDelete()` to remove a product from the database.

What students achieve: Full CRUD backend · Mongoose models with validation · API connected to MongoDB · MERN stack ready for deployment

Made with GAMMA

GUIDE

Getting Started with MongoDB (Local + Shell + Web UI)

Step 1 & 2: Download & Install

- Visit official website & download **Community Server**
- Choose **Complete Setup** during installation
- Enable **Install MongoDB as a Service**

Outcome: Installer ready & successfully installed.

Step 3, 4 & 5: Verify & Connect

- Verify: `mongod --version`
- Start Server: `mongod` (Port: 27017)
- Open Shell: `mongosh`

Outcome: Connected via terminal shell.

Step 6 & 7: DB & Collection

- Select DB: `use studentDB`
- Create Collection: `db.createCollection("students")`

Outcome: Schema structure initialized.

Steps 8-11: Core CRUD Operations

```
// INSERT & READ
db.students.insertOne({ name: "Parth", age: 21 })

db.students.find().pretty()
// UPDATE & DELETE
db.students.updateOne({ name: "Parth" }, { $set: { age: 22 } })

db.students.deleteOne({ name: "Amit" })
```

Advanced Tools: Use **MongoDB Compass** (GUI) at `localhost:27017` or **MongoDB Atlas** for Cloud deployment.

Made with GAMMA

PRACTICE

Hands-on: MongoDB Shell CRUD

Master the fundamentals of database interaction by creating a database and performing core operations directly through the MongoDB Shell (mongosh).

Task: Implement CRUD Operations

Follow these steps to build your database:

- **CREATE:** Initialize a new MongoDB database and insert your first document.
- **READ:** Query the collection to retrieve and filter data.
- **UPDATE:** Modify existing records using atomic operators.
- **DELETE:** Remove specific documents from your collection.

```
// 1. Create & Insert
db.users.insertOne({ name: "Alice", age: 25 })

// 2. Read
db.users.find({ name: "Alice" })

// 3. Update
db.users.updateOne({ name: "Alice" }, { $set: { age: 26 }
})

// 4. Delete
db.users.deleteOne({ name: "Alice" })
```

Goal: Navigate the mongosh environment · Understand JSON-like BSON storage · Practice real-time data manipulation

Made with GAMMA

ACTIVITY

Practice: Build a Mini Product API

Objective: Create a backend API using Node.js + Express + MongoDB + Mongoose to perform CRUD operations.
Problem: Build an API for managing products in an e-commerce system.

Step 1: Setup (5m)

- Create folder & init
- npm install express mongoose

```
npm init -y
npm i express mongoose
```

Step 2: Connect (5m)

```
mongoose.connect('.../productDB')
.then(()=>console.log("Connected"))
.catch(err=>console.log(err));
```

Step 3: Schema & Model (10m)

```
const productSchema = new mongoose.Schema({
name: String,
price: Number
});

module.exports = mongoose.model('Product',
productSchema);
```

Step 4: API Routes (10m)

```
app.post('/add-product', async (req, res) => {
const product = new Product(req.body);
await product.save();
res.send("Product Added");
});
```

Expected Output & Testing

Server: http://localhost:3000
Test via Postman: POST /add-product, GET /products

Evaluation Criteria

- DB Connection active
- Model & Schema correctness
- Functional POST & GET routes
- Data persistence verified

Extension

Add 'category' field and implement price sorting:

`Product.find().sort({ price: -1 })`

Outcome: Build a fully functional mini-backend with database persistence in under 30 minutes.

Made with GAMMA

Made with GAMMA